Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Využitie veľkých jazykových modelov v programovaní

Bakalárska práca

2026
Vladyslav Bilichenko

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Využitie veľkých jazykových modelov v programovaní

Bakalárska práca

Študijný program:	Informatika
Študijný odbor:	Informatika
Školiace pracovisko:	Katedra informatiky
Školiteľ:	tit. Meno Priezvisko, tit.

Bratislava, 2026
Vladyslav Bilichenko



Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta:
Študijný program:

Študijný odbor:
Typ záverečnej práce:
Jazyk záverečnej práce:
Sekundárny jazyk:

Názov:

Anotácia:

Vedúci:
Katedra:
Vedúci katedry:

Dátum zadania:

Dátum schválenia:

garant študijného programu

.....

študent

.....

vedúci práce

Čestné vyhlásenie: Čestne vyhlasujem, že celú bakalársku prácu na tému „Využitie veľkých jazykových modelov v programovaní“, vrátane všetkých jej príloh a obrázkov, som vypracoval/vypracovala samostatne, a to s použitím literatúry uvedenej v priloženom zozname.

Pri príprave tejto práce boli tiež použité nástroje umelej inteligencie [ZOZNAM NÁSTROJOV] za účelom [DÔVOD]. Nástroje umelej inteligencie som použil/použila v súlade s príslušnými právnymi predpismi, akademickými právami a slobodami, etickými a morálnymi zásadami za súčasného dodržania akademickej integrity. Som si vedomý/vedomá, že plne zodpovedám za správnosť výsledného textu.

Poďakovanie: Tu môžete poďakovať školiteľovi, prípadne ďalším osobám, ktoré vám s prácou nejako pomohli, poradili, poskytli dáta a podobne.

Abstrakt

Slovenský abstrakt v rozsahu 100-500 slov, jeden odstavec. Abstrakt stručne sumarizuje výsledky práce. Mal by byť pochopiteľný pre bežného informatika. Nemal by teda využívať skratky, termíny alebo označenie zavedené v práci, okrem tých, ktoré sú všeobecne známe.

Kľúčové slová: jedno, druhé, tretie (prípadne štvrté, piate)

Abstract

Abstract in the English language (translation of the abstract in the Slovak language).

Keywords:

Obsah

Úvod **1**

1 Analýza problému a teoretické východiská **3**
1.1 Veľké jazykové modely a Prompt Engineering 3
1.2 Obmedzenia kontextového okna 3
1.3 Retrieval-Augmented Generation (RAG) 4
1.4 Hodnotenie kvality vygenerovaného kódu 4
1.5 Optimalizácia promptu: Diskrétny problém batoha 5
1.6 Súvisiaca literatúra a príbuzné práce 5
1.6.1 Metódy výberu a kompresie kontextu 6
1.6.2 Hodnotenie užitočnosti znalostí 6
1.6.3 Zameranie doplňujúceho kontextu 7
1.6.4 Porovnanie s prístupom LongCodeZip 7

2 Návrh riešenia a architektúra systému **11**
2.1 Celkový prehľad systému 11
2.2 Schéma spracovania 12
2.3 Použité technológie 12

3 Implementácia **15**
3.1 Správa súborov a ukladanie stavov 15
3.2 Integrácia jazykového modelu a čistenie výstupu 15
3.3 Bezpečné spúšťanie Java kódu (Sandbox) 16
3.4 Dynamické programovanie pre optimalizáciu kontextu 16

4 Experimenty a Vyhodnotenie **19**
4.1 Metodika A/B testovania 19
4.2 Výsledky experimentu 19
4.3 Analýza a diskusia 20

Záver **21**

Literatúra **23**

Príloha A **27**

Zoznam obrázkov

1.1 Architektúra a proces spracovania kontextu v systéme LongCodeZip [17]. 9

2.1 Architektúra modulárneho spracovateľského reťazca pre optimalizáciu promptu. Nakreslené manuálne, preložené pomocou gemini 12

Zoznam tabuliek

4.1 Porovnanie úspešnosti generovania kódu (Základný prístup vs. optimalizované zadanie) 20

Úvod

V posledných rokoch sme svedkami masívneho rozvoja umelej inteligencie, pričom veľké jazykové modely (Large Language Models – LLM) začali radikálne transformovať mnohé odvetvia vrátane softvérového inžinierstva. Nástroje založené na LLM dokážu nielen asistovať pri písaní zdrojového kódu, ale aj navrhovať architektonické riešenia či refaktorovať existujúce programy. Kvalita a presnosť týchto výstupov však nie je garantovaná a kriticky závisí od kvality vstupného zadania – promptu.

Jednou z metód, ako zvýšiť úspešnosť jazykových modelov pri riešení špecifických programovacích úloh, je obohatenie promptu o relevantný kontext z odbornej literatúry. Tento prístup však naráža na technický limit modelov, ktorým je obmedzená veľkosť kontextového okna. Keďže do promptu nie je možné vložiť celú knihu, vzniká optimalizačný problém: ako vybrať tie najužitočnejšie úryvky textu tak, aby sme maximalizovali šancu na vygenerovanie správneho kódu a zároveň neprekročili povolený počet tokenov.

Cieľom tohto ročníkového projektu je návrh a implementácia automatizovaného softvérového nástroja (sprácovateľský reťazec), ktorý túto optimalizáciu vykonáva analyticky a deterministicky. Nástroj extrahuje znalosti z programátorskej literatúry, empiricky ohodnotí ich užitočnosť pomocou kompilácie a testovania vygenerovaného kódu v izolovanom prostredí a následne aplikuje algoritmus na riešenie problému batoha (Knapsack problem) pre zostavenie optimálneho promptu.

Text práce je štruktúrovaný do piatich hlavných kapitol. **Druhá kapitola** analyzuje teoretické východiská, obmedzenia jazykových modelov a matematický základ optimalizácie. **Tretia kapitola** sa venuje celkovému návrhu a architektúre vytvoreného modulárneho systému. **Štvrtá kapitola** detailne popisuje konkrétnu softvérovú implementáciu, správu stavov a bezpečné spúšťanie vygenerovaného kódu. **Piata kapitola** prezentuje metodiku a výsledky záverečného A/B experimentu, v ktorom porovnávame úspešnosť modelu so základným zadaním a s optimalizovaným kontextom. Prácu uzatvára **Záver**, ktorý zhrňuje dosiahnuté výsledky a navrhuje možnosti ďalšieho rozvoja projektu.

Kapitola 1

Analýza problému a teoretické východiská

Táto kapitola sa zaoberá teoretickým pozadím využitia veľkých jazykových modelov (LLM) pri generovaní zdrojového kódu. Rozoberáme v nej limity súčasných modelov, metódy na hodnotenie vygenerovaného kódu a matematický prístup k optimalizácii kontextu prostredníctvom algoritmu diskrétnej optimalizácie.

1.1 Veľké jazykové modely a Prompt Engineering

Veľké jazykové modely ako GPT-3.5, GPT-4 [3] alebo modely rodiny Claude predstavujú prelom v oblasti umelej inteligencie a spracovania prirodzeného jazyka. Sú trénované na rozsiahlom korpuse textových dát vrátane zdrojových kódov vo viacerých programovacích jazykoch, vďaka čomu dokážu nielen rozumieť textu, ale aj generovať syntakticky a sémanticky korektný kód.

Kvalita výstupu, ktorý model vygeneruje, však kriticky závisí od vstupného zadania, tzv. promptu. *Inžinierstvo dopytov* je disciplína zameraná na návrh a optimalizáciu týchto vstupov. V kontexte programovania nestačí len popísať požadovanú funkcionalitu. Presnosť riešenia sa výrazne zvyšuje, ak model v prompte obdrží aj špecifické pravidlá, obmedzenia architektúry alebo ukážky správneho kódu.

1.2 Obmedzenia kontextového okna

Každý jazykový model má pevne stanovený limit na množstvo textu, ktoré dokáže spracovať v rámci jednej interakcie. Toto obmedzenie sa nazýva kontextové okno (context window) a meria sa v tokenoch (fragmentoch slov alebo znakov). Pre model GPT-3.5-turbo je základné kontextové okno obmedzené, hoci novšie modely (napríklad o200k_base pre GPT-4o) ponúkajú podstatne väčšiu kapacitu.

Pri snahe poskytnúť modelu ako kontext celú odbornú knihu (napr. *Effective Java* [2]) rýchlo narazíme na tento limit. Prekročenie limitu vedie k chybe API, zatiaľ čo príliš dlhý kontext, ktorý sa zmestí do maxima, môže spôsobiť fenomén známy ako *Stratený v strede*, kedy model ignoruje informácie umiestnené v strede rozsiahleho promptu. Z tohto dôvodu je nevyhnutné do promptu vyberať len tie najrelevantnejšie fragmenty textu.

1.3 Retrieval-Augmented Generation (RAG)

Prekonanie obmedzení kontextového okna a zamedzenie halucinácií modelu sa v modernej praxi rieši pomocou architektúry RAG (Generovanie rozšírené o vyhľadávanie) [10]. Tradičný prístup RAG funguje na báze vektorového vyhľadávania, kedy sa na základe otázky užívateľa nájdu najpodobnejšie dokumenty v databáze a tie sa pridajú do promptu.

V rámci nášho ročníkového projektu pristupujeme k tomuto konceptu inovatívne. Namiesto sémantického vyhľadávania vytvárame deterministický pipeline, ktorý apriórne (pred samotným testovaním) zmeria užitočnosť jednotlivých fragmentov literatúry a na základe týchto empirických metrík poskladá optimálny kontext.

1.4 Hodnotenie kvality vygenerovaného kódu

Aby sme vedeli kvantifikovať, aký užitočný bol daný fragment literatúry pre model, musíme objektívne zhodnotiť vygenerovaný kód. Využívame na to dataset programovacích úloh (v našom prípade benchmark *HumanEval* [4] prispôsobený pre jazyk Java).

Kvalitu hodnotíme na dvoch úrovniach:

- **Úspešnosť kompilácie (Compilation Rate):** Overuje, či model vygeneroval syntakticky správny kód, ktorý prejde kompilátorom (`javac`) bez chýb.

- **Úspešnosť testov (Test Pass Rate):** Prísnejšia metrika, ktorá spúšťa vygenerovaný kód voči vopred pripraveným testovacím prípadom (jednotkové testy) a kontroluje logickú správnosť riešenia.

Súčet úspešných testov pre daný fragment textu na vzorke úloh nám dáva empirickú hodnotu (váhu), ktorá vyjadruje "kvalitu"danej teoretickej rady.

1.5 Optimalizácia promptu: Diskrétny problém batoha

Po ohodnotení všetkých fragmentov literatúry stojíme pred optimalizačnou úlohou: ako z obrovského množstva textu vybrať takú podmnožinu, ktorá maximalizuje súhrnnú užitočnosť, ale jej celková veľkosť neprekročí limit kontextového okna (v tokenoch).

Tento problém je ekvivalentný klasickému *0/1 Knapsack Problem* (Problém batoha) [6]. Matematicky ho môžeme definovať nasledovne: Máme množinu n fragmentov literatúry, kde každý fragment má:

- **Hodnotu** v_i (úspešnosť testov, Test Pass Rate),
- **Váhu** w_i (veľkosť fragmentu v tokenoch).

Našou úlohou je nájsť binárny vektor $x = (x_1, x_2, \ldots, x_n) \in \{0,1\}^n$, ktorý určuje, či je i-ty fragment zahrnutý v prompte ($x_i = 1$) alebo nie ($x_i = 0$), tak, aby sme maximalizovali účelovú funkciu:

$$\max \sum_{i=1}^{n} v_i x_i \qquad \text{Vzorec 1}$$

za podmienky kapacitného obmedzenia kontextového okna W:

$$\sum_{i=1}^{n} w_i x_i \leq W \qquad \text{Vzorec 2}$$

Tento problém v projekte riešime pomocou dynamického programovania, čím zabezpečíme nájdenie globálneho optima. Asymptotická časová zložitosť tohto prístupu je $O(nW)$, čo je pre veľkosti modelov a množstvo textu postačujúce a garantuje nám optimálny kontext (optimálny prompt) pre následné testovanie.

1.6 Súvisiaca literatúra a príbuzné práce

Problematika výberu a optimalizácie kontextu pre veľké jazykové modely predstavuje v súčasnosti vysoko aktuálnu výskumnú oblasť. Existujúce prístupy sa snažia zmierniť obmedzenia kapacitného limitu modelov a eliminovať nadbytočné informácie, pričom sa však v prevažnej miere spoliehajú na aproximačné metódy a sémantické posudzovanie textu, ako napríklad kosínusovú podobnosť vektorových vnorení alebo prekrývanie kľúčových slov [9] [14] [18] [19]. Naša práca sa od publikovaných riešení zásadne odlišuje v dvoch kľúčových aspektoch: vo formulácii problému ako exaktného 0/1 problému batoha riešeného dynamickým programovaním a v empirickom hodnotení užitočnosti textu na základe úspešnej kompilácie a vykonania programového kódu.

Súvisiaci výskum možno rozdeliť do troch hlavných kategórií:

1.6.1 Metódy výberu a kompresie kontextu

Väčšina súčasných prác pristupuje k zostavovaniu vstupného kontextu ako k maximalizácii submodulárnej funkcie, čo predstavuje výpočtovo náročnú optimalizačnú úlohu. Z dôvodu zníženia tejto zložitosti autori štandardne volia rôzne aproximačné a chamtivé postupy. Publikácie ako [5,9,15] využívajú techniky postupného výberu (napríklad metódu výberu najlepšej predpony alebo lenivý chamtivý výber). Tieto metódy nachádzajú uplatnenie pri skracovaní odpovedí externých nástrojov pre autonómne softvérové systémy [15], pri správe histórie dialógu a dlhodobej pamäte [9], alebo pri selekcii vzorových ukážok pri učení priamo v kontexte [7,8].

Pri zohľadnení pevného rozpočtu na dĺžku vstupu viaceré práce [16] uplatňujú adaptívne chamtivé stratégie, zatiaľ čo iní autori navrhujú evolučné a genetické algoritmy penalizujúce nadmerný rozsah textu [20]. Špecifické postavenie majú práce [18,19], kde je výber formulovaný ako exaktná binárna úloha celočíselného lineárneho programovania, a [17], kde autori aplikujú algoritmus problému batoha pomocou dynamického programovania, avšak len lokálne na úrovni blokov kódu vo vnútri vybraných funkcií.

Naša odlišnosť: Na rozdiel od riešení založených na submodulárnych aproximáciách považujeme pravidlá extrahované z odbornej literatúry za logicky ucelené a nezávislé jednotky. Úlohu formulujeme ako klasický diskrétny 0/1 problém batoha a riešime ju pomocou deterministického dynamického programovania. Oproti prístupu v práci [17] neobmedzujeme optimalizáciu iba na izolované vnútorné bloky programových funkcií, ale vykonávame globálny výber pravidiel naprieč celým zdrojovým dokumentom, vďaka čomu garantujeme nájdenie absolútneho globálneho optima bez nutnosti kompromisných heuristík.

1.6.2 Hodnotenie užitočnosti znalostí

Tradičným prístupom k hodnoteniu prínosu jednotlivých úryvkov textu je využitie sémantickej podobnosti a vektorových reprezentácií. Autori bežne merajú kosínusovú podobnosť v priestore vektorových vnorení [14,19], prípadne ju kombinujú s textovým prekrývaním kľúčových výrazov [15]. V práci [5] autori trénujú neurónový enkodér na mapovanie vlastností do skrytého stavového priestoru a v [16] používajú matice podobnostných jadier.

Iné metodiky merajú užitočnosť prostredníctvom štatistických charakteristík modelu, napríklad podmienenej perplexity [17], alebo prostredníctvom lexikálnej zhody: sledovaním prekrytia n-gramov [8] či overovaním prítomnosti presného reťazca cieľovej odpovede vo vnútri kontextu [1]. Pri generovaní programov sa využíva aj miera štrukturálneho prekrytia abstraktných syntaktických stromov [7] alebo priame bodové vyhodnotenie zhody [20]. Práca [18] sa taktiež plne opiera o sémantické skóre z vektorovej databázy.

Naša odlišnosť: Vektorová podobnosť, lexikálna zhoda ani syntaktické prekrytie nedokážu objektívne posúdiť, či dané inžinierske odporúčanie skutočne pomôže modelu vytvoriť funkčný a korektný softvér. Náš návrh úplne odstraňuje závislosť od vektorových databáz a textovej zhody. Váhu jednotlivým fragmentom priraďujeme čisto empirickým experimentom: každý úsek textu je overený priamo v procese riešenia úloh a jeho hodnota vychádza striktne z toho, či vygenerovaný zdrojový kód úspešne prejde kompiláciou a sadou formálnych jednotkových testov.

Tento exaktný prístup so sebou prináša jasné výhody aj nevýhody. Hlavnou výhodou je oveľa presnejšie a objektívnejšie posúdenie užitočnosti daného textu, keďže sa nespoliehame na teoretickú sémantickú podobnosť, ale na reálnu funkčnosť vygenerovaného kódu. Na druhej strane, nevýhodou sú výrazne vyššie výpočtové a tokenové nároky počas prípravnej fázy, keďže každý fragment literatúry vyžaduje samostatné volanie LLM a vytvorenie izolovaného prostredia na kompiláciu.

1.6.3 Zameranie doplňujúceho kontextu

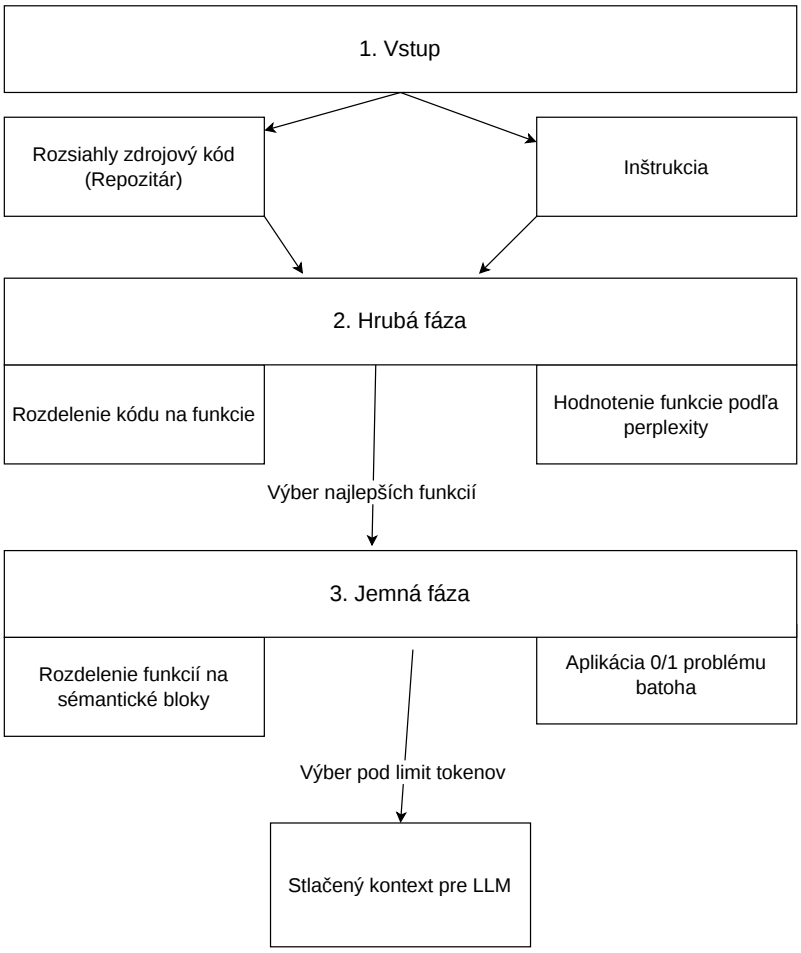
Významná časť súčasného výskumu sa sústreďuje na automatické dopĺňanie programového kódu priamo z existujúcich softvérových repozitárov. Tieto systémy sú často integrované priamo do vývojových prostredí s cieľom v reálnom čase radiť programátorom pri písaní kódu. Nástroje ako RepoGenix [12], REPOFUSE [11] či systém VITAL-RAG [13] prepájajú štrukturálny a analogický kontext zo zdrojových kódov projektu. Ďalšou bežnou aplikáciou RAG prístupov mimo priameho dopĺňania kódu je napríklad few-shot prompting, kde sa z databázy dynamicky vyberajú najvhodnejšie ukážkové príklady pre učenie modelu priamo v kontexte. Tieto metódy spravidla uplatňujú heuristické orezávanie a prideľovanie pevných limitov jednotlivým objektom s cieľom šetriť priestor v zadaní a znížiť odozvu modelu.

Naša odlišnosť: Náš prístup neslúži na vyhľadávanie a dopĺňanie existujúceho programového kódu z projektového úložiska. Sústredíme sa na obohatenie znalostnej bázy modelu o abstraktné princípy, metodiky a pravidlá písania kvalitného softvéru z odbornej literatúry. Namiesto dodávania hotových úryvkov kódu na priame vloženie optimalizujeme samotné metodické zadanie, čím model systematicky vedieme k návrhu syntakticky aj logicky správnych riešení.

1.6.4 Porovnanie s prístupom LongCodeZip

Blízkym prístupom k našej práci je metóda LongCodeZip [17], ktorá taktiež využíva algoritmus 0/1 problému batoha riešený dynamickým programovaním. Autori riešia problém dlhého kontextu pri dopĺňaní zdrojového kódu z rozsiahlych repozitárov. Ich systém funguje v dvoch fázach. V prvej, hrubej fáze, rozdelia kód na funkcie a ohodnotia ich pomocou podmienenej perplexity (ako dobre kontext predikuje inštrukciu).

V druhej, jemnej fáze, rozdelia vybrané funkcie na menšie sémantické bloky a práve na úroveň týchto blokov aplikujú 0/1 problém batoha na výber tých najrelevantnejších pod stanovený tokenový limit.Hoci zdieľame matematický aparát diskrétnej optimalizácie, naša práca sa zásadne líši v dvoch rovinách. Po prvé, LongCodeZip hodnotí užitočnosť blokov interne na základe perplexity jazykového modelu, zatiaľ čo my váhu určujeme čisto empiricky – úspešnou kompiláciou a prechodom cez jednotkové testy. Po druhé, LongCodeZip aplikuje dynamické programovanie iba lokálne (vo vnútri už predvybraných funkcií z repozitára), kým náš systém extrahuje abstraktné inžinierske pravidlá z literatúry a hľadá globálne optimálny výber naprieč celou vedomostnou bázou.



Obr. 1.1: Architektúra a proces spracovania kontextu v systéme LongCodeZip [17].

Kapitola 2

Návrh riešenia a architektúra systému

Táto kapitola popisuje celkovú architektúru nášho riešenia. Systém bol navrhnutý ako modulárny dátový spracovateľský reťazec (reťazec na spracovanie dát) v programovacom jazyku Python. Každý krok reprezentuje samostatný spustiteľný skript, ktorý číta dáta zo štandardizovaného formátu JSON, vykoná potrebnú transformáciu alebo ohodnotenie, a výsledok opäť uloží. Tento prístup zaručuje jednoduchú rozšíriteľnosť, ladenie a stabilitu pri výskyte chýb.

2.1 Celkový prehľad systému

Architektúra je rozdelená do štyroch hlavných logických fáz, ktoré na seba striktne nadväzujú:

1. **Spracovanie literatúry a príprava dát (Parsing & Tokenization):** Vstupom je odborná kniha vo formáte PDF (napríklad *Effective Java*). Pomocou knižnice `PyMuPDF` sa z dokumentu extrahuje surový text, odstránia sa prebytočné biele znaky a text sa rozdelí na menšie logické celky (odstavce alebo vety). Každý fragment dostane unikátny identifikátor (hash) a je mu vypočítaná presná veľkosť v tokenoch (pomocou knižnice `tiktoken` a kódovania `cl100k_base`).

2. **Generovanie a evaluácia kódu (Predictor & Evaluator):** V tejto fáze systém prechádza pripravené fragmenty textu. Pre každý fragment a pre každú programovaciu úlohu z datasetu sa zostaví izolovaný prompt. Tento prompt je odoslaný cez API do modelu GPT-3.5. Následne prichádza kľúčová časť: vygenerovaný Java kód je izolovaný, očistený od Markdown formátovania a spustený v sandboxe (pomocou modulu `subprocess`).

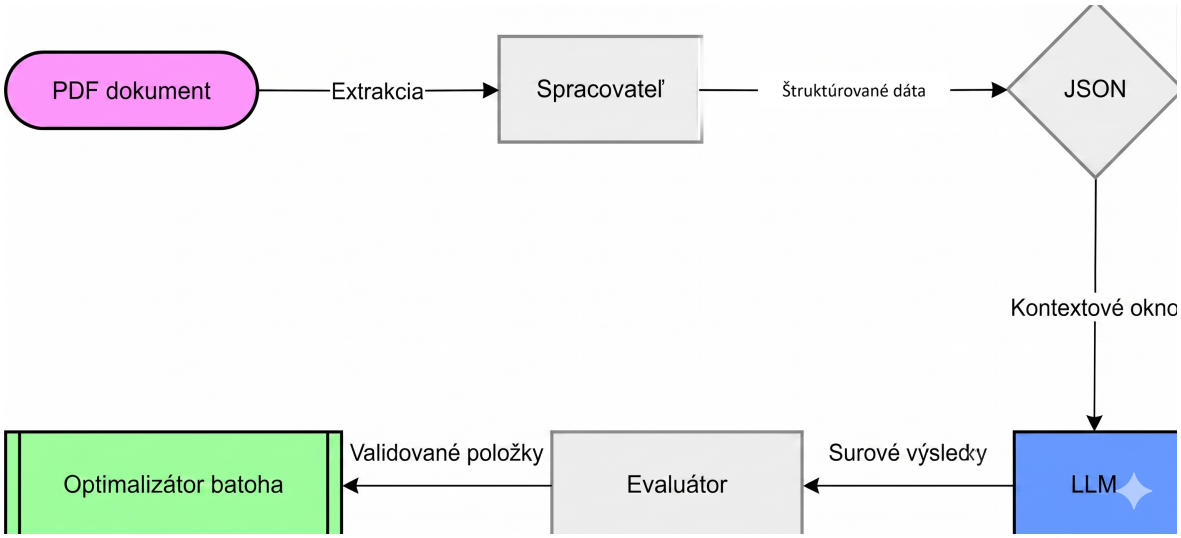
    - Najprv prebehne pokus o kompiláciu pomocou nástroja `javac`.
    - V prípade úspešnej kompilácie sa kód spustí (`java`) a overí sa voči testom z datasetu.

3. **Optimalizácia kontextu (Knapsack Solver):** Keď sú všetky fragmenty ohodnotené metrikou úspešnosti, spustí sa algoritmus dynamického programovania. Ten zo stoviek fragmentov vyberie presne tú kombináciu, ktorá sa zmestí do vopred definovaného limitu tokenov (napríklad 1200 tokenov) a zároveň maximalizuje súčet získaných bodov (tzv. Podiel úspešných testov).

4. **Finálny A/B Test (Final-Prompt Tester):** Vybrané fragmenty s najvyššou teoretickou hodnotou sú spojené do jedného veľkého kontextu, tzv. optimálneho promptu. Systém následne vygeneruje záverečný report, v ktorom porovná úspešnosť LLM pri riešení úloh úplne bez kontextu (Baseline) a s využitím nášho optimalizovaného výberu z literatúry.

2.2 Schéma spracovania

Pre lepšiu ilustráciu toku dát a interakcie medzi modulmi uvádzame zjednodušenú schému na obrázku 2.1.



Obr. 2.1: Architektúra modulárneho spracovateľského reťazca pre optimalizáciu promptu. Nakreslené manuálne, preložené pomocou gemini

2.3 Použité technológie

Na implementáciu vyššie popísaného riešenia sme zvolili nasledujúce technológie a nástroje:

- **Jazyk:** Python 3.10+ (pre riadiacu logiku) a Java (pre kompiláciu a beh vygenerovaného kódu).

- **Rozhranie LLM:** Knižnica `openai` s architektonickou prípravou pre integráciu open-source modelov prostredníctvom abstrakčnej vrstvy `LiteLLM`.
- **Spracovanie dát:** Vlastná trieda `JSONHandler` postavená nad štandardnou knižnicou `json` pre perzistenciu stavov.

Kapitola 3

Implementácia

Táto kapitola sa zamerava na konkrétne detaily softvérovej implementácie navrhnutej architektúry. Kód bol písaný s dôrazom na čitateľnosť, modularitu a princípy objektovo-orientovaného programovania v jazyku Python.

3.1 Správa súborov a ukladanie stavov

Pre zabezpečenie nezávislosti jednotlivých krokov spracovateľkého reťazcu sme sa rozhodli nevyužívať tradičnú relačnú databázu, ale stavový systém založený na formáte JSON. Trieda `JSONHandler` udržiava dáta v pamäti a automaticky aktualizuje vnútorné indexy podľa stavu fragmentu (napr. *parsed*, *tokenized*, *evaluated*). Vďaka tomu dokáže každý spúšťací skript (*runner*) rýchlo načítať len tie dáta, ktoré sú pripravené pre danú fázu spracovania. O konzistentné cesty k súborom a logickú separáciu adresárov sa stará trieda `DirectoryManager`, ktorá dynamicky vyhľadáva koreňový adresár projektu.

3.2 Integrácia jazykového modelu a čistenie výstupu

Na komunikáciu s jazykovým modelom bola vytvorená trieda `Predictor`. V základnom nastavení využíva knižnicu `openai` na volanie modelu z rodiny GPT-3.5. Systém je však štrukturálne pripravený aj na integráciu iných poskytovateľov (napríklad lokálnych open-source modelov) cez rozhranie `LiteLLM`, čo zabezpečuje flexibilitu do budúcna bez nutnosti prepisovať hlavnú logiku.

Keďže jazykové modely majú tendenciu generovať kód obalený v Markdown syntaxi (napríklad ‘‘‘`java`), implementovali sme metódu `clean_llm_output`. Táto metóda deteguje a odstráni prebytočné formátovacie znaky a zabezpečí, že výstupom odovzdaným na evaluáciu je striktne čistý zdrojový kód.

3.3 Bezpečné spúšťanie Java kódu (Sandbox)

Najkritickejšou časťou evaluácie je kompilácia a spustenie kódu, ktorý vygenerovala umelá inteligencia. Keďže model môže vygenerovať kód obsahujúci nekonečné cykly alebo vážne syntaktické chyby, museli sme zaistiť bezpečné prostredie pre jeho beh.

Trieda `Evaluator` tento problém rieši vytvorením dočasného adresára `temp_java`, do ktorého zapíše vygenerovaný kód spolu s testovacou obálkou. Následne pomocou štandardnej Python knižnice `subprocess` inicializuje nové procesy operačného systému:

- Spustí kompilátor `javac` s časovým limitom 15 sekúnd.
- V prípade úspechu spustí `java Main` s časovým limitom 10 sekúnd.

Ak proces presiahne stanovený čas (*Timeout*) alebo skončí chybou (nenulový návratový kód kompilácie), `Evaluator` bezpečne proces ukončí a fragmentu priradí nulové skóre pre daný test.

3.4 Dynamické programovanie pre optimalizáciu kontextu

Srdcom výberu ideálneho kontextu je trieda `KnapsackSolver`. Keďže hľadáme optimálne riešenie diskrétneho problému 0/1 batoha, využívame prístup dynamického programovania. Tabuľka stavov (`dp`) sa iteratívne napĺňa podľa pravidla, či sa aktuálny fragment (s váhou v tokenoch) zmestí do dočasného limitu kontextového okna a či jeho pridanie zvýši celkovú metriku úspešnosti.

Ukážka jadra algoritmu v jazyku Python je uvedená v listingu 3.1:

Algoritmus 3.1: Implementácia algoritmu Knapsack pomocou dynamického programovania

```
dp = [[0.0] * (W + 1) for _ in range(n + 1)]

for i in range(1, n + 1):
    item = items[i-1]
    weight = item["metadata"]["tokens"]["cl100k_base"]
    value = item["metadata"]["metrics"][self.metric_name] or 0.0

    for w in range(W + 1):
        if weight <= w:
            dp[i][w] = max(dp[i-1][w], dp[i-1][w - weight] + value)
        else:
            dp[i][w] = dp[i-1][w]
```

Po naplnení matice systém prejde tabuľku spätne od konca a identifikuje presné identifikátory (`ID`) fragmentov textu, ktoré tvoria optimálny výsledok. Tento prístup je plne deterministický a na rozdiel od heuristických metód garantuje nájdenie globálneho optima vzhľadom na zvolený cieľ (napr. maximalizácia `test_pass_rate`).

Zdrojové kódy k implementácii a experimentom sú verejne dostupné v repozitári na službe GitHub: `https://github.com/bilichenko0/llm_prompt_tuning`.

Kapitola 4

Experimenty a Vyhodnotenie

Táto kapitola prezentuje výsledky testovania navrhnutého systému. Cieľom experimentu bolo overiť hlavnú hypotézu projektu: či pripojenie algoritmicky vybraného kontextu z odbornej literatúry (optimálny prompt) zlepší schopnosť jazykového modelu riešiť programovacie úlohy v porovnaní so základným prístupom bez kontextu (Baseline).

4.1 Metodika A/B testovania

Pre objektívne vyhodnotenie sme implementovali triedu `FinalPromptTester`, ktorá vykonáva automatizované A/B testovanie. Experiment prebiehal za nasledovných podmienok:

- **Model:** `qwen3-coder-next` (s teplotou nastavenou na 0.2 pre deterministickejšie výstupy).
- **Dátová sada:** Vzorka 40 úloh zo sady HumanEval prispôsobenej pre jazyk Java.
- **Zdrojový kontext:** Optimalizovaný výber 9 fragmentov (spolu 1198 tokenov) z knihy *Effective Java* pomocou algoritmu batohu (s limitom 1200 tokenov).

Každá úloha bola modelom riešená dvakrát. V prvom prípade dostal model len zadanie úlohy (Baseline). V druhom prípade dostal rovnaké zadanie rozšírené o optimálny prompt. Oba vygenerované zdrojové kódy boli následne kompilované a testované voči rovnakej sade jednotkových testov.

4.2 Výsledky experimentu

Súhrnné výsledky experimentu sú zaznamenané v Tabuľke 4.1. Obe metriky (úspešnosť kompilácie aj úspešnosť prechodu testami) sú uvedené ako pomer úspešných pokusov k celkovému počtu úloh.

Tabuľka 4.1: Porovnanie úspešnosti generovania kódu (Základný prístup vs. optimalizované zadanie)

Prístup	**Úspešná kompilácia**	**Úspešné logické testy**
Základný prístup (bez kontextu)	39 / 40 (97,5 %)	38 / 40 (95,0 %)
Optimalizované zadanie (s kontextom)	40 / 40 (100,0 %)	40 / 40 (100,0 %)

4.3 Analýza a diskusia

Ako môžeme vidieť z výsledkov, experiment úspešne potvrdil hlavnú hypotézu projektu. Pripojenie algoritmicky vybraného kontextu z odbornej literatúry dokázalo zlepšiť schopnosti jazykového modelu a viedlo k bezchybným výsledkom.

Základný prístup bez dodatočného kontextu dosiahol veľmi vysokú úspešnosť (95,0 % pre logické testy a 97,5 % pre kompiláciu). Tento jav naznačuje, že model `qwen3-coder-next` má pre základné algoritmické úlohy dostatok vnútorných vedomostí. Zvyčajne sa v takýchto prípadoch prejavuje efekt stropu, kedy pridaný kontext nedokáže výraznejšie vylepšiť už takmer optimálny výkon.

Náš experiment však ukázal, že doplnenie zadania o cielené fragmenty z knihy *Effective Java* (konkrétne 9 vybraných úsekov textu s celkovou veľkosťou 1198 tokenov) pomohlo modelu prekonať tento strop a dosiahnuť 100 % úspešnosť v kompilácii aj v logických testoch. Pridané rady modelu poskytli chýbajúce detaily potrebné na vyriešenie úloh, v ktorých pôvodne zlyhal.

Tieto výsledky dokazujú plnú funkčnosť celej navrhnutej postupnosti spracovania. Systém dokázal úspešne analyzovať dokument, vybrať najvhodnejší kontext pomocou optimalizačného algoritmu, spojiť ho so zadaním a správne vyhodnotiť vygenerovaný kód.

Záver

Cieľom tohto ročníkového projektu bolo navrhnúť a implementovať automatizovaný systém na optimalizáciu promptov pre veľké jazykové modely v doméne programovania, s využitím externej odbornej literatúry.

Tento cieľ sa podarilo úspešne naplniť vytvorením modulárneho softvérového riešenia v jazyku Python. Zrealizovaný pipeline dokáže samostatne spracovať textové dokumenty (PDF), rozdeliť ich na logické fragmenty a ohodnotiť ich užitočnosť na základe úspešnosti vygenerovaného Java kódu v prísnom, izolovanom prostredí (sandbox). Následne systém využíva pokročilý algoritmus dynamického programovania (0/1 Knapsack Problem) na výber tých najlepších úryvkov tak, aby sa neprekročil limit kontextového okna jazykového modelu.

Finálny experiment na rozšírenej vzorke 40 úloh jednoznačne preukázal prínos navrhnutého riešenia. Zatiaľ čo model pri základnom prístupe dosiahol vysokú úspešnosť, pridanie optimalizovaného kontextu mu pomohlo prekonať pomyselný výkonnostný strop a vyriešiť všetky úlohy úplne bezchybne. Analýza tohto javu potvrdila, že automatizovaná optimalizácia zadania pomocou algoritmu batoha je efektívnym nástrojom na zlepšenie generovania kódu.

Vyvinutý systém je vysoko prispôsobiteľný a pripravený na ďalšie nasadenie. Vďaka integrácii vrstvy `LiteLLM` je možné do projektu jednoducho zapojiť rôzne (aj lokálne) jazykové modely. Práca tvorí pevný základ pre ďalší výskum, najmä v oblasti testovania komplexnejších programovacích úloh, kde by sa hlboké znalosti z odbornej literatúry mohli plne zhodnotiť.

Literatúra

[1] Ananto Nayan Bala. Recall is not enough: A reader-context diagnostic for budget-constrained retrieval-augmented generation. *arXiv preprint arXiv:2607.00725*, 2026.

[2] Joshua Bloch. *Effective java*. Addison-Wesley Professional, 2017.

[3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *CoRR*, abs/2005.14165, 2020.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021.

[5] Yu Chen, Ruishuo Chen, Xun Wang, Zhuoran Li, and Longbo Huang. Optimal skill selection for llm agents with provable bicriteria guarantees. *arXiv preprint arXiv:2608.19993*, 2026.

[6] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to algorithms*. MIT press, 2022.

[7] Yali Du, Hui Sun, and Ming Li. Post-incorporating code structural knowledge into pretrained models via icl for code translation. *IEEE Transactions on Software Engineering*, 51(11):3038–3055, 2025.

[8] Yali Du, San-Zhuo Xi, Hui Sun, and Ming Li. Design-specification tiling for icl-based cad code generation. *arXiv preprint arXiv:2603.12712*, 2026.

[9] Manu Ghulyani, Arunabh Singh, Karan Bharadwaj, Ankit Nath, and Suranjan Goswami. Pacms: Submodular context selection as a pluggable engine for llm agents. *arXiv preprint arXiv:2606.20047*, 2026.

[10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc.

[11] Ming Liang, Xiaoheng Xie, Gehao Zhang, Xunjin Zheng, Peng Di, Hongwei Chen, Chengpeng Wang, Gang Fan, et al. Repofuse: Repository-level code completion with fused dual context. *arXiv preprint arXiv:2402.14323*, 2024.

[12] Ming Liang, Xiaoheng Xie, Gehao Zhang, Xunjin Zheng, Peng Di, Wei Jiang, Hongwei Chen, Chengpeng Wang, and Gang Fan. Repogenix: Dual context-aided repository-level code completion with language models. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ASE '24, page 2466–2467, New York, NY, USA, 2024. Association for Computing Machinery.

[13] Zijian Lu, Yonghua Lu, Mingcai Chen, Yiping Zuo, Xin He, Weijun Wang, and Weibei Fan. Vital-rag: Invariance race for context allocation in coding agents. *arXiv preprint arXiv:2607.26937*, 2026.

[14] Chao Peng, Bin Wang, Zhilei Long, and Jinfang Sheng. Adagres: Adaptive greedy context selection via redundancy-aware scoring for token-budgeted rag. *arXiv preprint arXiv:2512.25052*, 2025.

[15] Tatiana Petrova, Andrei Mazniak, and Radu State. Agents don't paginate: First-chunk selection for llm tool responses. *arXiv preprint arXiv:2608.26130*, 2026.

[16] Khizar Qureshi, Geoffrey Martin, and Yifan Peng. Budget-aware routing for long clinical text. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Findings of the Association for Computational Linguistics: ACL 2026*, pages 42583–42598, San Diego, California, United States, July 2026. Association for Computational Linguistics.

[17] Yuling Shi, Yichun Qian, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. Longcodezip: Compress long context for code language models. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 141–153, 2025.

[18] Samuel Taiwo, Mohd Amaluddin Yusoff, and Kelechi Enyioha. Token-budget-aware context selection for rag via integer linear programming. In *2026 8th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (ICHORA)*, pages 1–5, 2026.

[19] Jonathan Tonglet, Manon Reusens, Philipp Borchert, and Bart Baesens. SEER : A knapsack approach to exemplar selection for in-context HybridQA. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13569–13583, Singapore, December 2023. Association for Computational Linguistics.

[20] Tom Zehle, Moritz Schlager, Timo Heiß, and Matthias Feurer. Capo: Cost-aware prompt optimization. *arXiv preprint arXiv:2504.16005*, 2025.

Príloha A: obsah elektronickej prílohy

V elektronickej prílohe priloženej k práci sa nachádza zdrojový kód programu a súbory s výsledkami experimentov. Zdrojový kód je zverejnený aj na službe GitHub: `https://github.com/bilichenko0/llm_prompt_tuning`.